

4.2 Vision Transformers

Image patches as tokens. The Week 3 block on pixels.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

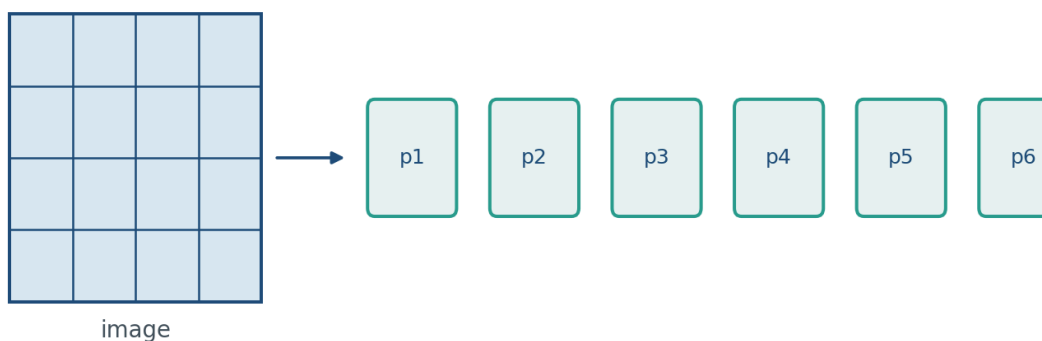
A **vision transformer (ViT)** is the Week 3 block on **patches** instead of words. Once an image is a sequence, every LLM trick (attention, positions, [CLS], later CLIP) applies.

1. Patches as tokens

Split a $H \times W$ image into $P \times P$ patches, flatten each patch, map with a linear layer to width d .

$N = HW / P^2$ tokens. Add a learned **[CLS]** if you classify, and add **positions** (row-column, or a 1-D index).

A ViT treats patches the way an LLM treats tokens

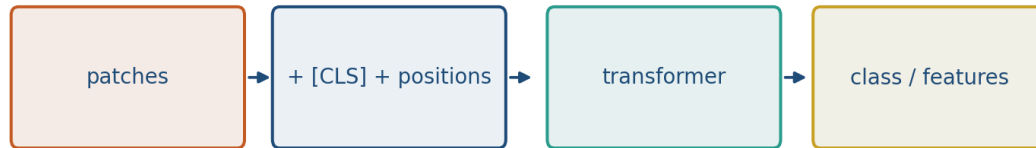


A CNN shares a small kernel and builds a hierarchy. A ViT sees global context in layer 1, at $O(N^2)$ cost. For a 224 image and $P = 16$, $N = 196$, which is a short paragraph.

Flattening a $16 \times 16 \times 3$ patch is 768 numbers — coincidentally GPT-2's d . The linear map is $E \in \mathbb{R}^{768 \times d}$ if you want a different width. RGB is just three extra channels in the flatten.

2. The same stack

Vision transformer: same block as language, different tokens



Dosovitskiy et al. (2021): ViTs need more data than ResNets if trained from scratch; they shine with scale. In this course you will almost always **start from a pretrained** ViT (CLIP’s image tower, or a Hugging Face checkpoint), not from random pixels.

Hybrid: a small CNN stem, then a transformer. Same idea.

Positions are not optional. Patch 1 vs patch 196 are different places on the photo. Attention without positions is a bag of tiles.

On the board, cut a 4×4 grid into four 2×2 tiles and number them 0–3 in row-major order. That numbering **is** the 1-D position. A 2-D (row, col) encoding is nicer for images but the course default is: add a vector per index, same as language.

CLIP’s image tower is often this ViT. When you later freeze “the vision encoder,” you are freezing patch embed + transformer. You are not freezing a ResNet unless the checkpoint says so.

3. Teaching this note

30–40 minutes. Count patches on a square image, flatten one toy 2×2 patch, add [CLS]+positions, then “same block as Week 3.” Play the **patch embedding** stretch of the Umar Jamil video (treat it as a ViT/VLM walkthrough even if the title is broader). Lab 4 will flatten patches in numpy; do one flatten on the board first.

Minute plan: 10 min $N = HW / P^2$ with a 4×4 grid; 10 min flatten + linear; 8 min [CLS] and positions; 10 min video on patches. Do not train a ViT in this block.

4. Worked example

Image $H = W = 4$, $P = 2$, 1 channel. Then $N = 4$ patches. Top-left patch

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

flattens to $\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$. A linear map $W \in \mathbb{R}^{2 \times 4}$ with, say, first row all 0.25, sends it to a 2-D token whose first coordinate is the mean 2.5.

Standard ImageNet: 224×224 , $P = 16$, $N = 196$. Plus [CLS] : **197** tokens. Attention map $197^2 = 38809$ scores per head.

Count for practice later: $384 / 16 = 24$, $24^2 = 576$ patches.

A smaller patch (say $P = 8$ on 224) gives $N = 784$ tokens, a longer “paragraph,” and a heavier N^2 map. That is why 16 is the default: enough spatial pieces, still a short sequence.

Hybrid reminder: a 3-layer CNN stem can map pixels to a coarser grid, then the transformer sees fewer tokens. Same $N = HW_{\text{feat}} / 1$ idea.

You do not implement ViT from random init in this course. You count patches, you add positions, you reuse Week 3.

5. Where students get stuck

- Using overlapping CNN-style windows and then miscounting $N = HW / P^2$.
 - Forgetting positions so the model cannot tell top from bottom.
 - Training a ViT from scratch on a tiny medical set.
-

6. Video

Watch [Umar Jamil: Vision Transformer / VLM walkthrough](#).

Use this URL even if the title is a broader VLM talk. Pause on **image patches as tokens**, the linear patch embedding, and positional encodings. That is the ViT; CLIP/BLIP wrap extra towers around it.

7. Practice

1. Count tokens for a 384×384 image, patch 16.
2. Why add positions if attention already sees all patches?
3. A medical image dataset with 800 scans: from-scratch ViT or a pretrained tower? Why?
4. For 224×224 , $P = 16$, include a [CLS] token. How many tokens enter the transformer? How many scores in one attention map?
5. A 32×32 RGB image, $P = 8$. Flatten one patch: how many numbers before the linear map? How many patches in the whole image?